



C++ Programming Language

Presented By

Usha Rani Sargurunathan

STG Group

Overview of C++

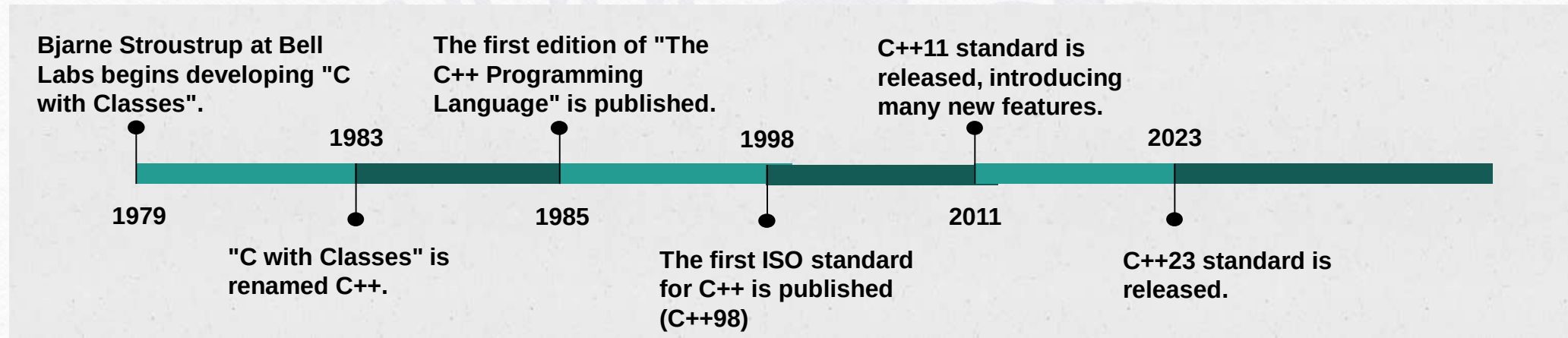
C++ is a high-performance, compiled, general-purpose programming language created by **Bjarne Stroustrup** in **1979** as an **extension of the C programming language**.

It prioritizes speed and runtime efficiency, C++ is the foundational language behind

- **Operating systems,**
- **Video game engines,**
- **Flight control software, and**
- **High-frequency trading platforms.**

In contrast to C, C++ supports classes and objects, while C does not.

History of C++



Setting Up a C++ Environment

- To write and run C++ code on your machine, you need two fundamental tools:
- **A Text Editor / IDE:** [Visual Studio Code](#), Microsoft Visual Studio, or [Code::Blocks](#).
- **A C++ Compiler:** Translates your plain text code into executable files (.exe or binary).
 - *Windows:* MinGW-w64 (GCC) or MSVC.
 - *macOS:* Clang (installed via Xcode Command Line Tools).
 - *Linux:* GCC (g++).

Setting Up a C++ Environment

Follow the steps below to install C++ on Windows:

- Install VS Code
- Download MinGW Compiler
- Run the Installer
- Add MinGW to System PATH
- Verify `g++ --version` in the command prompt
- Install C/C++ Extension in VS Code

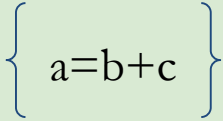
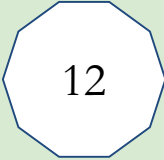
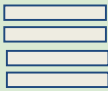
How Program works on
system ? How memory
is managed ?

Memory Layout

Why understanding memory layout is crucial ?

- ❖ To understand how programs store and access data.
- ❖ Essential for writing efficient and error-free code.
- ❖ Aids in debugging and troubleshooting memory-related issues (e.g., segmentation faults, stack overflows).

Memory Layout

Data Segments		Characteristics		Key Points
1	Code Segment (contains the compiled machine code of the program.)	Read-only Fixed Size		- Think of it as the program's "blueprint" or the set of instructions that the computer follows. - Stores the program's machine code.
2	Data Segment (Stores global and static variables.)	long-term data		- These variables exist throughout the entire program's execution. - Initialized Data Segment: explicitly initialized with a value. - Uninitialized Data Segment (BSS): These are typically initialized to zero by the system.
3	Heap (For dynamic memory allocation.)	Dynamic Size Manual Management		- Dynamic Size: The heap can grow or shrink as the program runs. - Think of it as a large pool of memory that the program can use when it needs extra space during runtime. - Requires careful management to avoid errors.
4	Stack (For function calls and local variables.)	LIFO Automatic Management		- When a function is called, a "stack frame" is created. - The stack frame contains the function's local variables, parameters, return address, and saved register values. - When the function finishes, the stack frame is removed (popped).

Worked example

```
int globalCount = 0; // Data Segment

void greet()
{
    // Code Segment (the function itself)
    int localAge = 25; // Stack
    Int *heapValue = new int(99); // 'heapValue' pointer on Stack,
    // the int(99) itself on Heap
}
```

If I call the same function three times, does it use the same stack space or new stack space each time?

If I allocate on the heap
and never delete it, does it
disappear when the
function ends?

Quick Check - To identify where each variable lives

```
int total = 100;
```

Data Segment

```
void process() {
```

```
    static int callCount = 0;
```

Data Segment

```
    int x = 5;
```

Stack Frame – x, *y

```
    int *y = new int(10);
```

Heap - *y's value

```
}
```

First C++ Program

"Hello World" Program

```
#include <iostream> // 1. Header file library for input/output

int main()
{
    // 2. Main function where execution starts
    // 3. Standard character output stream to print text
    std::cout << "Hello, World!" << std::endl;
    return 0;      // 4. Signals that the program finished successfully
}
```

Code Breakdown

- **#include <iostream>**: A preprocessor directive that imports standard input-output stream functionalities, enabling the use of `std::cout`.
- **int main()**: The entry point of every C++ program; execution always begins inside this function.
- **std::cout <<**: The standard output object used alongside the insertion operator (`<<`) to send text to the screen.
- **std::endl**: Flushes the stream buffer and inserts a newline character, moving the cursor down.
- **;**: Semicolons act as statement terminators, marking the end of a specific action.

Run Your First C++ Program

- First open VS Code, click on the File in the top menu and then select New File.
- Then, save this file with a .cpp extension by clicking on **File** again, then **Save As**, and type your filename ending in .cpp
- Now, write the following code into your file.
- Then click on the **run** button on the top right side of your screen.
- To run your C++ program on the **Terminal**:
 - `g++ hello.cpp -o hello`
 - `./hello`

Namespaces in Programming

Definition: A declarative block used to group related code (like classes, functions, and variables) under a **unique name**.

Purpose:

- To organize code into logical groups.
- To prevent name collisions between identifiers that have the same name but are defined in different parts of a program or different libraries.

Analogy: Think of it like a family's last name. Two people can have the same first name (e.g., John), but their full names (John Smith, John Doe) are unique because of their last names.

Without namespaces, defining two functions with the exact same name and signature causes a compile-time **"redefinition"** error.

Create and Use a Namespace

You declare a namespace using the **namespace keyword**, and you access its contents using the **scope resolution operator (::)**

Concept: Directly specifying the namespace along with the identifier using a scope resolution operator.

Syntax (Common in C++): `NamespaceName::IdentifierName`

E.g. `namespace nameOfSpace {
 // type1 member1
 void funcname() { //do the task}
}`

When to Use:

- When you need to explicitly refer to an item from a specific namespace to avoid ambiguity.
- For single, infrequent uses of an identifier from a namespace.

Example:

- `std::cout` (in C++)

Example NameSpace

```
#include <iostream>
namespace MyMath {
    double add(double a, double b) {
        return a + b;
    }
}
namespace YourMath {
    double add(double a, double b) {
        return a + b + 1.0; // A different implementation
    }
}
int main() {
    double result1 = MyMath::add(5.0, 3.0);
    double result2 = YourMath::add(5.0, 3.0);
    std::cout << "MyMath result: " << result1 << std::endl;
    std::cout << "YourMath result: " << result2 << std::endl;
    return 0;
}
```

```
MyMath result: 8
YourMath result: 9
```

Using Namespace with The using Directive

Concept: A directive that brings all the identifiers from a specified namespace into the current scope with the **using** keyword.

Syntax : `using namespace NamespaceName;`

When to Use:

- When you frequently use many identifiers from a particular namespace within a specific file or scope.
- Can make code more concise.

Caution: Can potentially reintroduce naming conflicts if two "used" namespaces contain identifiers with the same name.

Code Comparison - Without the **using** Directive

You must **explicitly state** where each item comes from every time you use it.

```
#include <iostream>
```

```
int main()
```

```
{
```

```
std::cout << "Hello, World!" << std::endl; // Requires std:: prefix
```

```
return 0;
```

```
}
```

Code Comparison - With the **using** Directive

By introducing the directive, the code becomes less verbose.

```
#include <iostream>
```

```
using namespace std; // The using directive
```

```
int main()
```

```
{
```

```
cout << "Hello, World!" << endl; // No std:: prefix needed
```

```
return 0;
```

```
}
```

Using Namespace with The using Declaration

Concept: A declaration that brings a *specific* identifier from a namespace into the current scope.

Syntax (Common in C++): `using NamespaceName::IdentifierName;`

When to Use:

- When you only need a few specific identifiers from a namespace.
- Provides a balance between full qualification and bringing in an entire namespace, reducing the chance of conflicts compared to the `using` directive.

Example:

- `using std::cout;`
- `using std::endl;`

Using Namespace with The using Declaration

```
#include <iostream>
using std::cout; // Only imports cout
using std::endl; // Only imports endl
int main()
{
    cout << "This is clean and safe!" << endl; // std::cin would still require the
    prefix because it wasn't imported

    return 0;
}
```

Using Namespace with The using Directive - Example

```
#include <iostream>
namespace Physics {
    double gravity = 9.81;
}
int main()
{
    using namespace Physics; // Brings everything from Physics into this local scope

    std::cout << "Gravity is: " << gravity << "\n"; // No prefix needed (physics::gravity)
    return 0;
}
```

Difference at a Glance

Using Directive	Using Declaration
<code>using namespace std;</code>	<code>using std::cout;</code>
Imports all members of a namespace	Imports only the specified member
Easier to write	More selective and safer
May cause name conflicts	Reduces name conflicts
Common in small programs	Preferred in large projects

Namespace Aliases

Concept: Creating a shorter, more convenient name (an alias) for a long or deeply nested namespace.

Syntax:

- `namespace MyAlias = OriginalLongNamespaceName;` (C++)

When to Use:

- To improve code readability when dealing with very long namespace names.
- To simplify access to deeply nested namespaces.

Real-World Practical Example:

- `namespace fs = boost::filesystem;`
- `fs::path myPath("main.cpp");` // Instead of: `boost::filesystem::path myPath("main.cpp");`

Namespace –using (example)

```
#include <iostream>

namespace a_very_long_and_complex_namespace_name {
    void do_work() {
        std::cout << "Working in a very long namespace." <<
std::endl;
    }
}

int main() {
    namespace MyShortName = a_very_long_and_complex_namespace_name;
    MyShortName::do_work();
    return 0;
}
```

Output: Working in a very long namespace.

Namespace Composition (Nested Namespaces)

Concept: Defining namespaces within other namespaces, creating a hierarchical structure.

Purpose:

- To further organize code into more granular, logical sub-groups.
- Reflects a more complex module structure.

Syntax:

- `namespace OuterNamespace { namespace InnerNamespace { /* code */ } } //Traditional`
- Or, in C++17 and later: `namespace OuterNamespace::InnerNamespace { /* code */ } //Modern`

Example:

- `Company::Product::Module::Class`

Benefits: Enhanced organization and reduced global scope pollution.

Example - Nested Namespaces

```
#include <iostream>
namespace Project {
    namespace Graphics {
        void draw() {
            std::cout << "Drawing graphics." << std::endl;
        }
    }
    namespace Audio {
        void play() {
            std::cout << "Playing audio." << std::endl;
        }
    }
}
int main() {
    Project::Graphics::draw();
    Project::Audio::play();
    return 0;
}
```

```
Drawing graphics.
Playing audio.
```

Example - Nested Namespaces

```
#include <iostream>
// Modern C++17 syntax
namespace Project::Graphics {
    void draw() {
        std::cout << "Drawing graphics." << std::endl; }}
namespace Project::Audio {
    void play() {
        std::cout << "Playing audio." << std::endl; }}
int main() {
    Project::Graphics::draw();
    Project::Audio::play();
    return 0;
}
```

```
Drawing graphics.
Playing audio.
```

C concepts used in C++

Operators

If else

Switch

Loops - for, while, do while

Break,continue,goto

Functions basics,types,recursion

Pointer basics

Structures and union

Concepts of Object Oriented



The picture can't be displayed.

In C, if you wanted to represent a car, what would you do?"

(a struct with fields, and separate functions that take the struct as a parameter).

Struct Example – Car 🚗

```
struct Car
{
    int speed;
    int fuel;
};

void accelerate(struct Car *c)
{
    c->speed += 10;
}
```

Problem in C

Nothing stops another part of the program from directly setting

`c->fuel = -500;`

which makes no physical sense.

The data and the rules protecting that data are **disconnected**.

That disconnect is the exact problem OOP solves.

What is OOP?

OOP stands for Object-Oriented Programming.

An object-oriented programming language uses **objects** in its programming.

Programming with object-oriented concepts aims to emulate real-world concepts such as inheritance, polymorphism, abstraction, etc., in a program.

C++ language was designed with the main intention of adding object-oriented programming to C language.

The main aim of OOP is to **bind together the data and the functions** that operate on them so that no other part of the code can access this data except that function.

Fundamentals of the Object-Oriented Approach

- Objects as fundamental building blocks.
- Emphasis on data over procedures.
- Programs organized around data and the functions that operate on it.
- Data hiding and protection.
- Communication through messages (method calls).
- Reusability of code through inheritance.

Why Objects?

- **Modeling Real-World Entities:** Objects naturally represent real-world concepts (e.g., a Car, a Bank Account, a Student).
- **Code Reusability:** Inheritance and composition allow existing code to be reused and extended.
- **Maintainability:** Encapsulation makes changes easier and less prone to side effects.
- **Scalability:** Modular design helps manage complexity in large systems.
- **Flexibility:** Polymorphism allows for flexible and extensible designs.
- **Reduced Complexity:** Abstraction simplifies the view of complex systems.

“A class is the blueprint.
An object is the house
built from it.”

Classes & Objects – The Different Perspectives

Class (Blueprint/Template):

- A blueprint or template for creating objects.
- Defines the common properties (attributes) and behaviors (methods) that all objects of that type will have.
- Does not occupy memory until an object is created from it.

Object (Instance/Entity):

- A concrete instance of a class.
- A real-world entity that exists in memory.
- Has its own unique state, identity, and behavior based on its class definition.
- Example: Car is a class, myRedCar is an object.

Class

C

```
struct Date
{
    int date;
    int month;
    int year;
};
```

C++

```
class Date
{
    public:
        int date;
        int month;
        int year;
        void printDate()
        {
            .....
        }
};
```

Access Specifier

Properties

Methods

Class Vs Structure

- Classes and structures are somewhat the same but still, they have some differences.
- For example, **we cannot hide data** in structures which means that everything is public and can be accessed easily, which is a major drawback of the structure because structures cannot be used where data security is a major concern. Another drawback of structures is that **we cannot add functions to them**.
- **Classes are user-defined data types** and are a template for **creating objects**. Classes consist of variables and functions which are also called **class members**.

Class Syntax

We use the **class** keyword to define a class in C++.

```
class class_name  
{  
    // body of the class  
};
```

Objects

Objects are **instances of a class**.

To create an object, we just have to specify the class name and then the object's name.

Objects can access class attributes and methods which are bound to the class definition.

It is recommended to put these attributes and methods in access modifiers so that their permissions can be better specified to allow them to be used by objects.

```
class class_name
{
    // body of the class
};
int main()
{
    class_name object_name;
    // object
}
```

Classes & Objects — the different perspectives

```
class Car {  
    private:  
        int speed;  
        int fuel;  
    public:  
        void accelerate() {  
            speed += 10; }  
};
```

```
Car myCar;      // one house built from the blueprintCar  
Car neighborsCar; // a second, independent house
```

Class and Object - Example

```
#include <iostream>
// Class declaration
class Point {
public:
    int x;
    int y;
};

int main() {
    // Object mechanism: creating instances of the Point class
    Point p1; // Creates an object 'p1'
    p1.x = 10;
    p1.y = 20;
    Point p2; // Creates another object 'p2'
    p2.x = 30;
    p2.y = 40;
    std::cout << "p1: (" << p1.x << ", " << p1.y << ")" << std::endl;
    std::cout << "p2: (" << p2.x << ", " << p2.y << ")" << std::endl;
    return 0;
}
```

Abstraction

As a driver, you press the accelerator. You don't need to know about fuel injectors or pistons.

You interact with a *simplified interface* that hides internal complexity.

In code: the class exposes `accelerate()`, and hides *how* speed is actually computed internally.

Encapsulation

The engine is under a locked hood.

You can't reach in and manually move a piston.

Data (fuel, engine internals) is *bundled with* the functions that are allowed to touch it, and protected from outside interference.

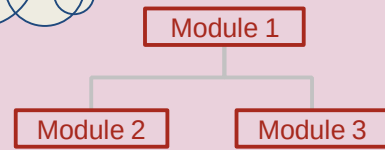
In code: private data members, public methods.

Modularity

The car is built from separate, independently-built modules —
engine, brakes, wheels —
each one can be designed, tested, and replaced independently.

In code: classes as independent units that compose together.

Breaking down a software system into smaller, independent, and interchangeable modules. Each module performs a specific function. Enhances maintainability, reusability, and collaboration.



Modularity

State, Identity, and Behavior

Every object has three defining aspects:

State (Attributes/Properties):

- The data or characteristics that define an object at a particular moment.
- Represented by variables within the object.
- Example: For a **Car** object, state includes **color**, **speed**, **fuel**.

Identity:

- The unique characteristic that distinguishes one object from another, even if they have the same state.
- Often represented by a memory address or a unique identifier.
- Example: Two **Car** objects can both be red and fast, but they are distinct entities.

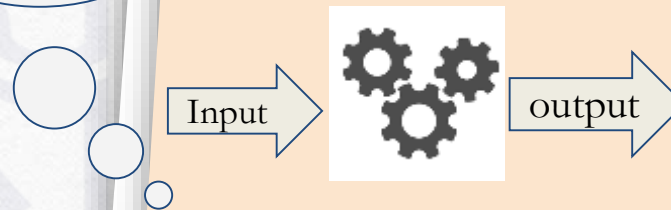
Behavior (Methods/Functions):

- The actions an object can perform or the operations that can be performed on it.
- Represented by functions or methods within the object.
- Example: For a **Car** object, behavior includes **accelerate()**, **brake()**.

State, Identity, and Behavior – Car example

- **State** = the current values of an object's data (speed = 60, fuel = 20)
- **Identity** = what makes this object *this specific one*, distinct from another object with identical state (two cars both going 60 with 20L fuel are **still two different cars**)
- **Behaviour** = what the object can *do* (accelerate(), brake())

It's achieved through **interfaces**, **abstract classes**, and well-defined **public methods**. You interact with an abstraction without needing to know the underlying code that makes it work.



Abstraction

Abstraction (Detailed)

Definition: The process of **hiding the internal implementation details** and **showing only the necessary features to the user**.

How it's achieved:

- **Abstract Classes:** Classes that cannot be instantiated directly and may contain abstract methods (methods without implementation).
- **Interfaces:** Purely abstract blueprints that define a set of methods that a class must implement.
- **Method Signatures:** Public methods expose functionality without revealing internal logic.

Benefits: Simplifies system design, reduces complexity, improves readability

Interface and Implementation

Interface

Defines what an object can do (its public methods and properties).

A contract that specifies the services an object provides.

Users interact with the interface without needing to know the internal workings.

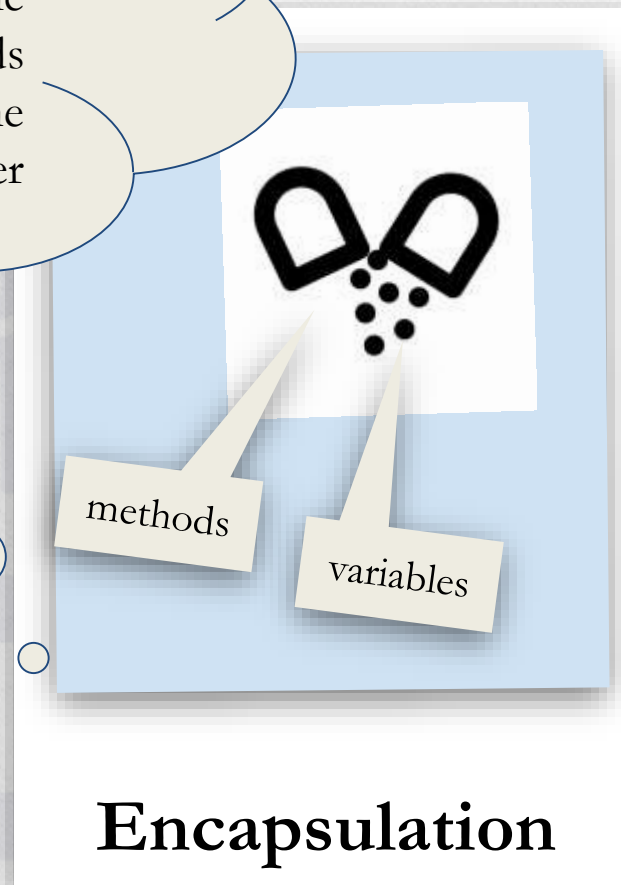
Implementation

The internal details of how an object performs its defined actions.

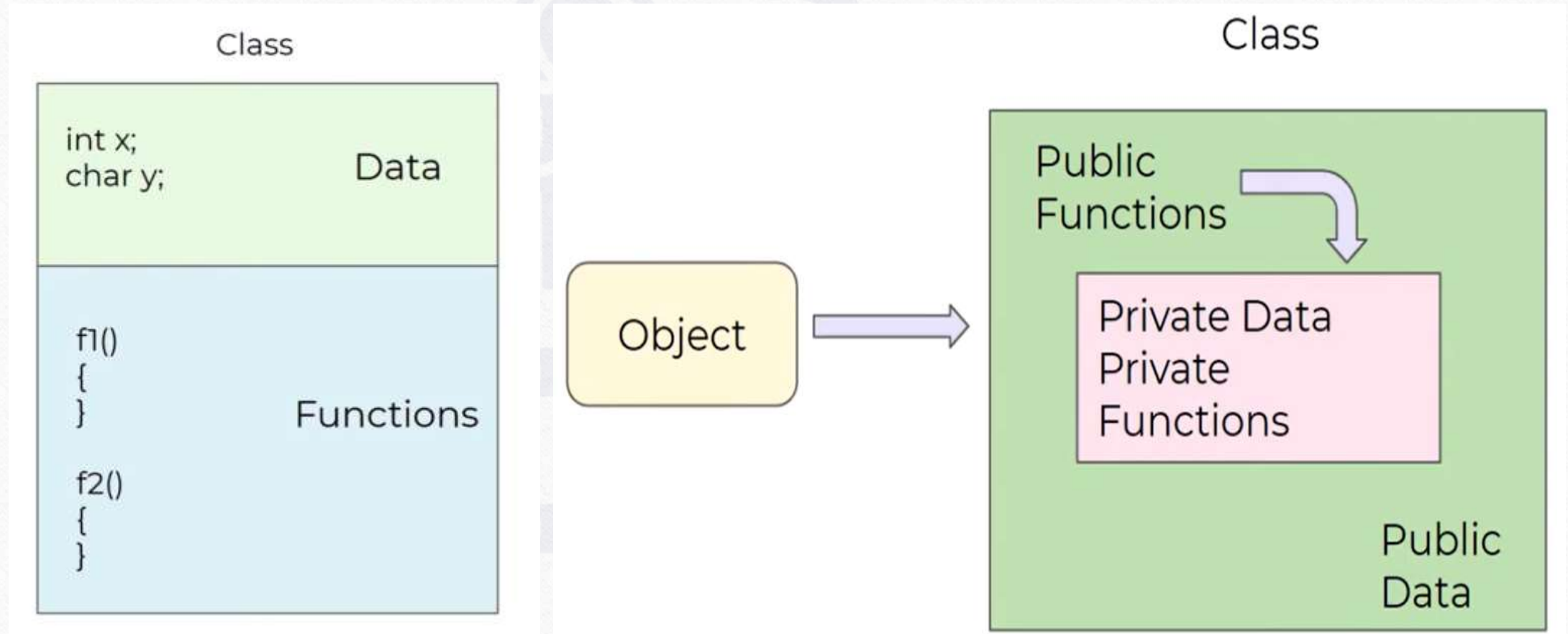
Hidden from the outside world (encapsulation).

Can change without affecting the interface or external code that uses it.

It's achieved through access modifiers (like `private`, `protected`). Private members are not directly accessible from outside the class. Public methods are provided to interact with the object's state in a controlled manner (getters and setters).



Encapsulation



Encapsulation (Detailed)

Definition: The mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit.

Data Hiding: Protecting the internal state of an object from unauthorized access or modification by external code.

Access Modifiers: Keywords like `public`, `private`, `protected` control visibility.

- `public`: Accessible from anywhere.
- `private`: Accessible only within the class.
- `protected`: Accessible within the class and its subclasses.

Benefits: Improves data integrity, enhances security, makes code more maintainable.

Encapsulation

- Binds data and code
- Acts like black box and protects data
- Public methods act as interface for accessing private data
- Enables us to write methods without any parameters as data and code are bound

Inheritance & Class Hierarchy

Extend the running example: SportsCar **is a** Car, with everything a Car has, plus more.

```
class SportsCar : public Car {  
    public:  
        void activateTurbo() { /* ... */ }  
};
```

"Inheritance lets you say 'this new thing is a specialized version of that existing thing' instead of rewriting everything from scratch."

Inheritance & Class Hierarchy

Inheritance:

- A mechanism where a new class (subclass/derived class) acquires the properties and behaviors of an existing class (superclass/base class).
- Promotes code reusability.
- Models "is-a" relationships (e.g., a **Dog** is a **Mammal**).

Class Hierarchy:

- A tree-like structure representing the relationships between classes through inheritance.
- Classes at higher levels are more general, while those at lower levels are more specific.
- Example: **Animal** -> **Mammal** -> **Dog** -> **GoldenRetriever**.

Polymorphism: Early v/s Late Binding

Imagine you have a Car and a SportsCar, and both have an `accelerate()` function, but they accelerate differently.

Polymorphism means you can call `accelerate()` on *either* one through a common interface, and the *correct* version runs automatically.

Early binding means the compiler decides which version to call at compile time.

Late binding means that decision happens at runtime instead.

Polymorphism: Early v/s Late Binding

Polymorphism (Many Forms):

- The ability of an object to take on many forms.
- Allows objects of different classes to be treated as objects of a common type.
- A single interface for multiple implementations.

Early Binding (Compile-time Polymorphism):

- Method calls are resolved at compile time.
- Achieved through **method overloading** (same method name, different parameters) and **operator overloading**.
- Faster execution as the call is fixed before runtime.

Late Binding (Runtime Polymorphism):

- Method calls are resolved at runtime.
- Achieved through **method overriding** (subclass provides a specific implementation for a method already defined in its superclass).
- Requires virtual functions/methods in some languages.
- More flexible and extensible

Object relationships

- **Is-a** (inheritance) — SportsCar is-a Car
- **Has-a** (composition) — Car has-a Engine(class contains an `Engine` object as one of its members, rather than inheriting from it)

```
class Engine { /* ... */ };
```

```
class Car {  
    private:  
        Engine engine; // Car "has-a" Engine  
};
```

Object Relationships

Association:

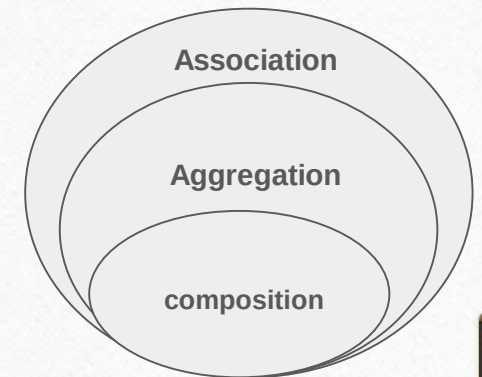
- A general **relationship between two independent classes**.
- Objects of one class are connected to objects of another.
- "Uses-a" relationship.
- Example: A **Student** *associates with* a **Course**.

Aggregation:

- A "has-a" relationship where **one object is part of another, but can exist independently**.
- A weaker form of association.
- Example: A **Department** *has* **Professors** (professors can exist without the department).

Composition:

- A "has-a" relationship **where one object is part of another and cannot exist independently**.
- A stronger form of aggregation.
- If the "whole" object is destroyed, the "part" objects are also destroyed.
- Example: A **House** *has* **Rooms** (rooms cannot exist without the house).



Task 1

1. Book — first class, first object

Create a class Book with data members

title (string),

author (string), and

pages (int).

Add a method displayInfo() that prints all three.

In main(), create one Book object, set its values, and call displayInfo().

Goal: basic class syntax, one object, one method call.

Solution

```
#include <iostream>
#include <string>
using namespace std;
class Book {
public:
    string title;
    string author;
    int pages;
    void displayInfo() {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Pages: " << pages << endl; } };

int main()
{
    Book b1;
    b1.title = "The Pragmatic
Programmer";
    b1.author = "Hunt & Thomas";
    b1.pages = 352;
    b1.displayInfo();
    return
}
```

Task 2

Rectangle — methods that compute, not just print

Create a class Rectangle with length and width.

Add methods `calculateArea()` and `calculatePerimeter()` that return values (not just print them).

In `main()`, create two Rectangle objects with different dimensions and print both objects' area and perimeter.

Goal: methods that return values; proving two objects hold independent data.

Solution

```
#include <iostream>
using namespace std;
class Rectangle {
public:
    double length;
    double width;

    double calculateArea() {
        return length * width; }
    double calculatePerimeter() {
        return 2 * (length + width); }
};

int main()
{
    Rectangle r1;
    r1.length = 5.0;
    r1.width = 3.0;
    Rectangle r2;
    r2.length = 10.0;
    r2.width = 2.5;
    cout << "r1 -> Area: " << r1.calculateArea()
        << ", Perimeter: " << r1.calculatePerimeter() <<
        endl;
    cout << "r2 -> Area: " << r2.calculateArea()
        << ", Perimeter: " << r2.calculatePerimeter() <<
        endl;
    return 0;
}
```

Task 3

BankAccount — encapsulation on purpose

Create a class BankAccount with a **private** member balance.

Add public methods deposit(amount), withdraw(amount), and checkBalance().

withdraw() must refuse the transaction (print an error, don't change balance) if the amount requested is more than what's available.

In main(), create one account, deposit some money, try to withdraw more than the balance, then withdraw a valid amount, and check the balance after each step.

```
#include <iostream>
using namespace std;
class BankAccount {
private:
    double balance;
public:
    void deposit(double amount) {
        balance += amount;
        cout << "Deposited: " << amount << endl; }
    void withdraw(double amount) {
        if (amount > balance) {
            cout << "Error: insufficient balance for withdrawal of “ <<
amount << endl;
        } else {
            balance -= amount;
            cout << "Withdrew: " << amount << endl; } }
    void checkBalance() {
        cout << "Current balance: " << balance << endl; } };
```

```
int main() {
    BankAccount acc;
    acc.deposit(1000);
    acc.checkBalance();

    acc.withdraw(5000); // should be
    refused
    acc.checkBalance();

    acc.withdraw(400); // should
    succeed
    acc.checkBalance();

    return 0;
}
```